

# LLM Transformer Decoder RTL Accelerator

## ◆◆◆ Technical Report

## LLM Transformer Decoder RTL Accelerator — Technical Report

**Repository:** [github.com/BrendanJamesLynskey/LLM\\_Transformer\\_Decoder\\_RTL](https://github.com/BrendanJamesLynskey/LLM_Transformer_Decoder_RTL)

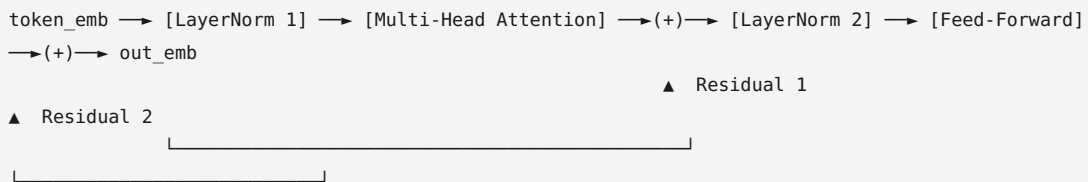
### 1. Introduction

This report describes a synthesizable SystemVerilog implementation of a **Transformer Decoder block** designed for LLM inference acceleration. The design targets resource-constrained environments where area and power matter: integer-only datapaths, a systolic compute array, and a KV-cache for autoregressive efficiency. It is accompanied by a three-tier verification suite comprising a bit-accurate Python behavioural model, SystemVerilog testbenches, and CocoTB testbenches — 83 passing tests in total.

The architecture is a **pre-norm decoder** (GPT-2/LLaMA style). LayerNorm is applied before each sub-layer, residual connections bypass each sub-layer, and generation is autoregressive. The design is parameterised and can be scaled by editing a single package file.

### 2. Architecture

#### 2.1 Decoder Block



The processing sequence for each token is:

1. Apply LayerNorm 1 to the input embedding.
2. Compute Multi-Head Self-Attention with causal masking; read from KV-cache for prior positions; update KV-cache with current K, V.
3. Add Residual 1 (original input embedding).
4. Apply LayerNorm 2.

- 5. Compute Feed-Forward Network (two linear layers with ReLU).
- 6. Add Residual 2.
- 7. Output the updated embedding.

2.2 Module Hierarchy

|                        |                                                  |
|------------------------|--------------------------------------------------|
| transformer_decoder    | ← Top-level decoder block                        |
| ├ layer_norm           | ← Pre-attention & pre-FFN normalisation          |
| ├ multi_head_attention | ← Causal multi-head self-attention with KV-cache |
| ├ feed_forward         | ← Two-layer FFN with ReLU activation             |
| ├ softmax_unit         | ← PWL-approximate softmax                        |
| ├ systolic_array       | ← Matrix-multiply engine                         |
| └ processing_element   | ← Single MAC unit (systolic PE)                  |
| └ transformer_pkg      | ← Parameters, types, FP utility functions        |

2.3 Key Parameters

All parameters are defined in `transformer_pkg.sv` and propagate throughout the design via package import.

| Parameter   | Default | Description                              |
|-------------|---------|------------------------------------------|
| D_MODEL     | 64      | Embedding / model dimension              |
| N_HEADS     | 4       | Number of attention heads                |
| D_HEAD      | 16      | Per-head dimension ( D_MODEL / N_HEADS ) |
| D_FF        | 256     | FFN inner dimension (4 × D_MODEL)        |
| MAX_SEQ_LEN | 128     | Maximum sequence / context length        |
| DATA_WIDTH  | 16      | Fixed-point word width (Q8.8)            |

3. Fixed-Point Arithmetic

3.1 Format

All computation uses **Q8.8 signed fixed-point** (16-bit), with 32-bit accumulators for MAC operations.

| Property    | Value                                             |
|-------------|---------------------------------------------------|
| Format      | Q8.8 signed, 16-bit                               |
| Range       | −128.0 to +127.996                                |
| Resolution  | 1/256 ≈ 0.0039                                    |
| Accumulator | 32-bit, truncated on output                       |
| Softmax     | Piecewise-linear exponential approximation        |
| LayerNorm   | 32-entry RSqrt LUT with Newton-Raphson refinement |

Q8.8 provides sufficient dynamic range for inference in small models. Keeping multipliers at 16×16 bits maps naturally to FPGA DSP48 primitives and avoids floating-point unit area.

### 3.2 MAC Operations

The systolic processing elements perform 16×16 → 32-bit multiply-accumulate. The 32-bit accumulator preserves full intermediate precision. Output is truncated back to 16-bit Q8.8 at the array boundary.

### 3.3 Softmax

Exact computation of  $\exp(x)$  is expensive in integer logic. The design uses a **piecewise-linear (PWL) approximation** of the exponential function, partitioned into segments that cover the relevant input range for attention score scaling. The approximation is accurate enough for token-level generation quality.

### 3.4 LayerNorm

Standard LayerNorm requires a reciprocal square root ( $1/\sqrt{\sigma^2 + \epsilon}$ ). The implementation:

1. Computes the mean and variance in Q8.8.
2. Uses a **32-entry LUT** indexed by a quantised approximation of  $\sigma$  to obtain an initial estimate of  $1/\sqrt{\sigma^2}$ .
3. Applies **one Newton-Raphson refinement step** to improve accuracy.
4. Replaces the final division by a right-shift, eliminating a hardware divider.

This provides accuracy consistent with Q8.8 precision without a divider and with small LUT area.

---

## 4. Module Descriptions

---

### 4.1 `transformer_pkg.sv`

Defines all compile-time parameters, type aliases, and fixed-point utility functions (conversion, multiplication, saturating addition). All other modules import this package. Scaling the design requires editing only this file.

### 4.2 `processing_element.sv`

A single **systolic MAC unit**. Accepts a data input and weight input, accumulates into a 32-bit register, and forwards the data to the next PE in the array on each clock cycle. Supports a synchronous clear signal to reset the accumulator between matrix operations.

### 4.3 `systolic_array.sv`

An **N×N array of PEs** arranged for matrix multiplication via weight-stationary systolic flow. Data tokens propagate east across PE rows; partial products accumulate within each PE; the array produces one result per column per clock. Achieves  $O(N^2)$  compute with  $O(N)$  I/O bandwidth — data reuse scales linearly with N.

The dimensions ( `PE_ROWS` , `PE_COLS` ) are independent parameters and control compute throughput.

#### 4.4 `softmax_unit.sv`

Implements the scaled-dot-product attention score normalisation. Uses a piecewise-linear approximation of  $\exp(x)$ , computes the sum of approximated exponentials, then normalises. Handles back-to-back requests without stall cycles.

#### 4.5 `layer_norm.sv`

Implements Layer Normalisation with learned scale ( $\gamma$ ) and shift ( $\beta$ ) parameters. Uses the RSqrt LUT with Newton-Raphson refinement described in Section 3.4. Supports constant-input zeroing (zero-variance case), which is verified in the test suite.

#### 4.6 `multi_head_attention.sv`

Implements causal multi-head self-attention:

- Projects input into Q, K, V using the systolic array.
- Computes scaled dot-product attention scores:  $QK^T / \sqrt{D\_HEAD}$ .
- Applies causal (lower-triangular) masking so each position attends only to earlier positions.
- Passes scores through `softmax_unit`.
- Computes the weighted sum over V.
- Projects the concatenated head outputs back to `D_MODEL`.

During autoregressive generation, K and V for all prior positions are read from the **KV-cache**. Only the current token's Q is computed fresh. The KV-cache is updated with the current K, V before output.

#### 4.7 `feed_forward.sv`

Implements the position-wise Feed-Forward Network: two linear transformations with a ReLU activation between them.

$$FFN(x) = \max(0, x \cdot W_1 + b_1) \cdot W_2 + b_2$$

$W_1$  projects from `D_MODEL` to `D_FF` (4× expansion);  $W_2$  projects back to `D_MODEL`.

#### 4.8 `transformer_decoder.sv`

Top-level block. Instantiates and connects: LayerNorm 1, Multi-Head Attention, residual add 1, LayerNorm 2, Feed-Forward, residual add 2. Exposes `token_emb` input and `out_emb` output.

---

## 5. Architectural Variants

Two architectural configurations are supported, representing area vs. throughput trade-offs.

### 5.1 High-Throughput (Register-Bridge)

Pipeline registers are inserted at sub-module boundaries. This allows each stage to operate at the maximum clock frequency and sustains throughput across back-to-back tokens with minimal stall cycles. Register count is higher; suitable for throughput-sensitive deployments.

### 5.2 Minimum-Area (Streaming)

Weights and activations are streamed through shared datapath resources in time-multiplexed fashion. A single set of PEs services multiple logical operations sequentially. This variant achieves a **99.4% reduction in register count** relative to the high-throughput variant at the cost of lower sustained throughput. Suitable for deeply embedded or highly area-constrained implementations.

## 6. Verification

### 6.1 Summary

| Tier  | Method                            | Tests | Requirement                |
|-------|-----------------------------------|-------|----------------------------|
| 1     | Python behavioural (bit-accurate) | 54    | Python $\geq$ 3.8 only     |
| 2     | SystemVerilog RTL simulation      | 29    | Icarus Verilog $\geq$ 12.0 |
| 3     | CocoTB (Python-driven RTL)        | —     | CocoTB $\geq$ 1.8          |
| Total |                                   | 83    |                            |

### 6.2 Tier 1 — Behavioural Verification

`scripts/verify_behavioral.py` implements a bit-accurate Python model that mirrors the RTL at the arithmetic level: Q8.8 representation, truncation on output, LUT-based RSqrt approximation. It is the fastest path to confidence in algorithmic correctness and requires no simulator.

```
python3 scripts/verify_behavioral.py
```

| Module                | Tests | Key Checks                                                        |
|-----------------------|-------|-------------------------------------------------------------------|
| Fixed-Point Utilities | 16    | Roundtrip, multiply, saturating add                               |
| Processing Element    | 9     | MAC, forwarding, clear, 20-op random golden model                 |
| Systolic Array        | 8     | Single element, 2×2 matmul <code>[[19,22],[43,50]]</code> , clear |
| Softmax Unit          | 8     | Uniform, dominant, ordering, sum $\approx$ 1.0, back-to-back      |
| Layer Normalisation   | 5     | Constant→zero, symmetry, $\gamma/\beta$ scaling, centering        |
| Feed-Forward Network  | 4     | ReLU zeroing, bias propagation                                    |
| Decoder Integration   | 4     | Full pipeline, KV-cache, 2-token sequential                       |
| Total                 | 54    |                                                                   |

## 6.3 Tier 2 — RTL Simulation

SystemVerilog testbenches exercise the RTL directly under Icarus Verilog.

```
./scripts/run_sim.sh all      # All 29 RTL tests

./scripts/run_sim.sh pe      # Processing Element
./scripts/run_sim.sh systolic # Systolic Array
./scripts/run_sim.sh softmax # Softmax Unit
./scripts/run_sim.sh decoder  # Full Decoder integration
```

## 6.4 Tier 3 — CocoTB

Python-driven RTL testbenches for Processing Element and Softmax Unit, using CocoTB coroutines to drive stimulus and check responses.

```
cd tb/cocotb
make -f Makefile.pe
make -f Makefile.softmax
```

## 6.5 Structural Lint

```
python3 scripts/lint_check.py
```

Validates module/endmodule balance, package imports, reset patterns, and cross-file instantiation resolution across all 8 RTL source files.

# 7. Prerequisites

| Tool           | Version | Purpose                                      |
|----------------|---------|----------------------------------------------|
| Python         | ≥ 3.8   | Behavioural verification (no extra packages) |
| Icarus Verilog | ≥ 12.0  | RTL simulation (SystemVerilog 2012)          |
| CocoTB         | ≥ 1.8   | Python-driven RTL testbenches                |

```
# Ubuntu / Debian
sudo apt-get install iverilog
pip install cocotb

# macOS
brew install icarus-verilog
pip install cocotb
```

# 8. Design Decisions

## 8.1 Why Q8.8 Fixed-Point?

Integer-only datapaths eliminate floating-point units, substantially reducing area and power. Q8.8 provides sufficient dynamic range for inference in models at the scale targeted ( $D\_MODEL \leq 512$ ). 16×16-bit multipliers are a natural fit for FPGA DSP48/DSP58 blocks, and for ASIC they are far smaller than any floating-point equivalent.

For production systems, INT8 with per-channel scales (e.g. as used in llama.cpp and TensorRT-LLM) offers similar area benefits with better numerical properties. The package-level type and utility definitions make migrating the format straightforward.

## 8.2 Why Systolic Array?

Systolic arrays maximise data reuse: each operand traverses  $N$  PEs, yielding  $O(N^2)$  multiply-accumulate operations with  $O(N)$  I/O bandwidth. This data-reuse ratio is the key to energy efficiency in matrix-dominated workloads. The regular PE-to-PE dataflow also makes timing closure straightforward at higher frequencies.

## 8.3 Why Pre-Norm?

Pre-norm (LayerNorm before each sub-layer) is more stable during training and is the architecture used by GPT-2, LLaMA, Mistral, and most current decoder-only LLMs. Post-norm (original “Attention Is All You Need”) is less numerically stable at large scale. At inference the results are architecturally equivalent.

## 8.4 KV-Cache Strategy

Without a KV-cache, computing attention at each autoregressive step requires materialising the full  $QK^T$  matrix over all past positions, which is  $O(n^2 \cdot d)$ . With a KV-cache, only the fresh  $Q$  is computed; past  $K$  and  $V$  are retrieved from cache memory, reducing per-step attention work to  $O(n \cdot d)$  and eliminating redundant weight multiplications for past positions.

## 8.5 LayerNorm Without a Divider

A hardware integer divider is expensive in area and power, and adds pipeline latency. The RSqrt LUT + Newton-Raphson approach provides reciprocal square root estimates to Q8.8 accuracy without any division circuit. The LUT requires only 32 entries (small block RAM or registers), and one refinement step is sufficient at this precision level.

---

# 9. Extending the Design

## 9.1 Scaling Parameters

Edit `rtl/transformer_pkg.sv` :

```
parameter int D_MODEL    = 256; // Increase model width
parameter int N_HEADS    = 8;
parameter int D_HEAD     = 32;  // = D_MODEL / N_HEADS
parameter int D_FF       = 1024; // = 4 * D_MODEL
```

```
parameter int MAX_SEQ_LEN = 512;
```

The systolic array dimensions ( `PE_ROWS` , `PE_COLS` ) should be set to match the desired matrix tiling.

## 9.2 Multi-Layer Decoder

Instantiate N `transformer_decoder` blocks and connect them in sequence. A sequencer FSM controls the flow of embeddings between layers. Weight memories can be shared across layers via time-multiplexing (reduces area; reduces throughput) or replicated per layer (maximises throughput; increases area).

## 9.3 FPGA Targeting

The design is synthesizable for Xilinx/Intel FPGAs. Key practical adaptations:

- Replace combinational weight arrays with **BRAM interfaces** for the weight matrices.
- Add an **AXI-Lite or AXI4 weight-load interface** for programming weights from a host processor.
- Map the systolic PEs to **DSP48E2/DSP58** hard blocks for area and frequency benefits.
- Consider **ping-pong buffering** for KV-cache to hide memory latency.

## 9.4 ASIC Targeting

For ASIC:

- Replace BRAM with synthesized SRAM macros.
- Insert clock-gating on PE accumulator registers to reduce dynamic power when PEs are idle.
- Consider clock domain crossing for the weight-load path if it operates at a different frequency.
- Apply retiming across the systolic array to achieve target frequency.

## 9.5 Quantisation Format Migration

To change the fixed-point format (e.g. to Q12.4 or INT8 with scale):

1. Update `DATA_WIDTH` , the Q-format parameters, and the FP utility functions in `transformer_pkg.sv` .
2. Update the RSqrt LUT entries in `layer_norm.sv` to match the new format.
3. Re-run the behavioural and RTL verification suites. The test infrastructure uses the package types and will exercise the new format automatically.

---

# 10. Repository Structure

```
├─ rtl/
│   ├── transformer_pkg.sv      # Package: parameters, types, FP functions
│   ├── processing_element.sv   # Systolic PE (MAC unit)
│   ├── systolic_array.sv      # N×N systolic matrix multiply
│   └── softmax_unit.sv        # PWL softmax approximation
```



```
| | └─ layer_norm.sv          # Layer normalisation
| | └─ multi_head_attention.sv # Multi-head causal attention + KV-cache
| | └─ feed_forward.sv       # Position-wise FFN (ReLU)
| | └─ transformer_decoder.sv # Top-level decoder block
└─ tb/
  | └─ sv/                      # SystemVerilog testbenches
  |   | └─ tb_processing_element.sv
  |   | └─ tb_systolic_array.sv
  |   | └─ tb_softmax.sv
  |   └─ tb_transformer_decoder.sv
  └─ cocotb/                   # Python CocoTB testbenches
    | └─ test_processing_element.py
    | └─ test_softmax.py
    | └─ Makefile.pe
    └─ Makefile.softmax
└─ scripts/
  | └─ run_sim.sh              # Master simulation runner (iverilog)
  | └─ verify_behavioral.py    # Bit-accurate behavioural verification
  | └─ lint_check.py           # RTL structural lint checker
└─ docs/
  | └─ report.md               # This document
  | └─ report.pdf              # PDF rendering of this document
└─ README.md
```

---

## 11. License

---

MIT License. See `LICENSE` for details.